

1번 문제

1) 시저 암호와 비제네르 암호를 설명하고 두 암호의 차이점을 기술하시오.

1-가) 시저 암호와 비제네르 암호의 암호화 방법을 설명하시오.

시저 암호는 평문의 각 알파벳을 일정한 거리만큼 밀어서 다른 알파벳으로 치환하는 단일 치환 방식의 암호이다. 시저 암호 체계에서 암호문의 i 번째 글자 C_i 는 i 번째 평문 P_i 와 비밀키 k 에 대해 다음과 같은 수식에 의해 정의된다.

$$C_i = (P_i + k) \pmod{26}$$

비제네르 암호는 평문의 각 알파벳을 반복되는 키워드에 따라 서로 다른 거리만큼 밀어서 치환하는 다중 치환 방식의 암호이다. 비제네르 암호 체계에서 암호문의 i 번째 글자 C_i 는 i 번째 평문 P_i , 비밀키 K , 비밀키의 길이 m 에 대해 다음과 같은 수식에 의해 정의된다.

$$C_i = (P_i + K_{i \pmod{m}}) \pmod{26}$$

1-나) 시저 암호와 비제네르 암호를 이용한 암호문의 알파벳 빈도 분포 차이점을 서술하시오.

시저 암호 체계에서 평문 알파벳 집합과 암호문 알파벳 집합을 잇는 암호화 함수는 일대일대응이다. 따라서 임의의 평문 알파벳 P 와 그에 대응하는 암호문 알파벳 C 에 대해 평문에서의 P 의 빈도는 암호문에서 C 의 빈도와 같다. 이러한 특성으로 인해 시저 암호 체계에서 평문의 알파벳 빈도 분포와 암호문의 알파벳 빈도 분포는 동형이다.

반면, 비제네르 암호 체계에서는 시저 암호 체계와 달리 평문 알파벳이 위치에 따라 서로 다른 암호문 알파벳이 될 수 있다. 따라서 임의의 평문 알파벳 P 에 대응하는 암호문 알파벳 C 가 P 의 위치에 따라 가변적으로 결정되므로, 평문에서 특정 알파벳이 집중되어 있던 빈도가 암호문의 여러 문자로 분산되는 빈도 평탄화 현상이 발생한다. 이러한 특성으로 인해 비제네르 암호문의 알파벳 분포는 평문의 분포와 동형성을 유지하지 못하며, 비밀키의 길이 m 이 커질수록 이산 균등 분포에 가까워진다.

2) “ciphertext1.txt”, “ciphertext2.txt” 각각에 대해 영어 자연어와의 일치지수(Index of Coincidence, IC)를 계산하고, 이를 이용하여 암호문을 시저 암호와 비제네르 암호로 구분하시오.

2-가) “ciphertext1.txt”, “ciphertext2.txt”의 일치지수를 각각 계산하시오. 이때, 일치지수는 소수 넷째자리에서 반올림하여 소수점 셋째자리 까지 표기한다.

문제에서 주어진 일치지수의 정의는 다음과 같으며, n_i 는 암호문에서 알파벳 i 의 출현 횟수, N 은 암호문 전체 길이이다.

$$IC = \frac{\sum_{i=A}^Z n_i(n_i - 1)}{N(N - 1)}$$

해당 정의를 활용하여 “ciphertext1.txt” 와 “ciphertext2.txt”의 일치지수를 구하면 아래와 같다.

```

1  # 1번문제 2-가) 풀이 코드
2  from collections import Counter
3
4  def read_ciphertext(filepath):
5      with open(filepath, 'r', encoding='utf-8') as f:
6          content = f.read()
7          return ''.join(c for c in content.upper() if c.isalpha())
8
9  def calc_ic(text):
10     N = len(text)
11     freq = Counter(text)
12     ic = sum(n * (n - 1) for n in freq.values()) / (N * (N - 1))
13     return ic
14
15  ct1 = read_ciphertext(r'.\ciphertexts1.txt')
16  ct2 = read_ciphertext(r'.\ciphertexts2.txt')
17
18  ic1 = calc_ic(ct1)
19  ic2 = calc_ic(ct2)
20
21  print(f"ciphertexts1.txt : {ic1:.3f}")
22  print(f"ciphertexts2.txt : {ic2:.3f}")

```

코드 1

```

ciphertexts1.txt : 0.068
ciphertexts2.txt : 0.044

```

그림 1. 코드 1 실행 결과

2-나) 계산된 일치지수 값을 이용하여 각 암호문의 암호 방식을 특정하고 그 근거를 설명하시오.

1-나)에서 살펴본 바와 같이 시저 암호문의 알파벳 빈도 분포는 평문의 알파벳 빈도 분포와 동형이고 비제네르 암호문의 알파벳 빈도 분포는 그렇지 않다. 알파벳 빈도 분포가 같다면 두 분포의 IC도 같다. 따라서 문제에서 주어진 [표1]에서 주어진 알파벳의 출현 빈도를 활용하여 구한 IC와 비슷한 IC를 가진 암호문은 시저 암호, 그렇지 않은 암호문은 비제네르 암호라고 할 수 있다.

```

24  # 1번문제 2-나) 풀이 코드
25  ENGLISH_FREQ = {
26      'A':8.17, 'B':1.50, 'C':2.78, 'D':4.25, 'E':12.70, 'F':2.23, 'G':2.02, 'H':6.09,
27      'I':6.97, 'J':0.15, 'K':0.77, 'L':4.03, 'M':2.41, 'N':6.75, 'O':7.51, 'P':1.93,
28      'Q':0.10, 'R':5.99, 'S':6.33, 'T':9.06, 'U':2.76, 'V':0.98, 'W':2.36, 'X':0.15,
29      'Y':1.97, 'Z':0.07
30  }
31
32  ic3 = sum((freq / 100) ** 2 for freq in ENGLISH_FREQ.values())
33  print(f"표1 : {ic3:.3f}")

```

코드 2

```

표1 : 0.066

```

그림 2. 코드 2 실행 결과

“ciphertexts1.txt”와 표1의 IC가 근사하므로 “ciphertexts1.txt”는 시저 암호임을, “ciphertexts2.txt”는 그렇지 않으므로 비제네르 암호임을 알 수 있다.

3. 시저 암호로 분류된 암호문의 평문을 복원하여라.

3-가) ‘2번’ 문제에서 시저 암호로 분류된 암호문에 대하여, 가능한 모든 키 값을 이용하여 암호문을 복호화하시오. 그리고 각 키로 복호화된 결과에서 알파벳(A-Z)별 출현 빈도를 계산하시오.

'2번' 문제에서 시저 암호로 분류된 암호문은 "ciphertext1.txt"이다. 해당 암호문에 대하여 가능한 키는 0부터 25까지이고 해당 키를 활용한 복호화 결과와 알파벳별 출현 빈도는 다음과 같다.

```

1  # 1번문제 3-가) 풀이 코드
2  from collections import Counter
3  import os
4
5  def read_ciphertext(filepath):
6      with open(filepath, 'r', encoding='utf-8') as f:
7          content = f.read()
8          return ''.join(c for c in content.upper() if c.isalpha())
9
10 def caesar_dec(text, k):
11     return ''.join(chr((ord(c)-65-k)%26+65) for c in text if c.isalpha())
12
13 os.makedirs('caesar_results', exist_ok=True)
14
15 ct1 = read_ciphertext(r'.\ciphertexts1.txt')
16
17 for k in range(26):
18     decrypt = caesar_dec(ct1, k)
19     freq = Counter(decrypt)
20     N = len(decrypt)
21     freq_pct = {c: round(cnt / N * 100, 2) for c, cnt in sorted(freq.items())}
22     print(f"key : {k}\nfreq : {freq_pct}\ndecrypt : {decrypt[:40]}")
23     with open(f'caesar_results/key_{k:02d}.txt', 'w', encoding='utf-8') as f:
24         f.write(f"key : {k}\n")
25         f.write(f"freq : {freq_pct}\n")
26         f.write(f"decrypt : {decrypt}\n")

```

코드 3

```

key : 0
freq : {'A': 2.15, 'B': 0.91, 'C': 0.67, 'D': 0.23, 'E': 1.78, 'F': 0.29, 'G': 8.42, 'H': 2.31, 'I': 3.77, 'J': 3.69, 'K': 12.69, 'L': 2.49, 'M': 1.23, 'N': 4.52, 'O': 7.46, 'P': 0.19, 'Q': 0.18, 'R': 5.11, 'S': 2.51, 'T': 7.17, 'U': 7.43, 'V': 2.23, 'W': 0.14, 'X': 6.02, 'Y': 6.84, 'Z': 9.57}
decrypt : ZNKIUTYZOZAZOUTULZNXKXVAHROIULQUXKGINGVZ
key : 1
freq : {'A': 0.91, 'B': 0.67, 'C': 0.23, 'D': 1.78, 'E': 0.29, 'F': 8.42, 'G': 2.31, 'H': 3.77, 'I': 3.69, 'J': 12.69, 'K': 2.49, 'L': 1.23, 'M': 4.52, 'N': 7.46, 'O': 0.19, 'P': 0.18, 'Q': 5.11, 'R': 2.51, 'S': 7.17, 'T': 7.43, 'U': 2.23, 'V': 0.14, 'W': 6.02, 'X': 6.84, 'Y': 9.57, 'Z': 2.15}
decrypt : YMJHTSXNYNZYNTSTKYMJWJUZGQNHKPTWJFHMFLUY
key : 2
freq : {'A': 0.67, 'B': 0.23, 'C': 1.78, 'D': 0.29, 'E': 8.42, 'F': 2.31, 'G': 3.77, 'H': 3.69, 'I': 12.69, 'J': 2.49, 'K': 1.23, 'L': 4.52, 'M': 7.46, 'N': 0.19, 'O': 0.18, 'P': 5.11, 'Q': 2.51, 'R': 7.17, 'S': 7.43, 'T': 2.23, 'U': 0.14, 'V': 6.02, 'W': 6.84, 'X': 9.57, 'Y': 2.15, 'Z': 0.91}

```

그림 3. 코드 3 실행 결과의 일부 : key가 0,1,2인 경우의 알파벳별 출현 빈도와 복호화된 결과의 일부이다.

모든 키에 대한 복호화 결과와 알파벳별 빈도는 첨부된 ceaser_results 폴더에서 확인할 수 있다.

3-나) [표 1]에서 제시된 영어 알파벳 출현 빈도(기대값)를 기준으로, '3-가'에서 구한 알파벳별 출현 빈도에 대하여 카이제곱(Chi-squared) 적합도 검정을 수행하시오. 그리고, 해당 값을 이용하여 키로 추측되는 값을 결정하시오.

문제에서 주어진 카이제곱 적합도 검정 수식은 아래와 같다. 이때 $freq(P_i)$ 는 암호문을 복호화한 후 알파벳 i 의 출현 횟수이며, $freq(N_i)$ 는 [표 1]을 이용한 알파벳 i 의 예상 출현 횟수이다.

$$X^2(k) = \sum_{i=A}^Z \frac{(freq(P_i) - freq(N_i))^2}{freq(N_i)}, P_i = DEC(C_i, k)$$

복호화된 평문의 알파벳 빈도 분포는 [표 1]에서 제시한 알파벳 빈도 분포와 차이가 거의 없다. 따라서 모든 키에 대해 카이제곱 적합도 검정 값을 구하고 그 중 가장 작은 값을 만드는 k 를 키로 추측할 수 있다.

```

28 # 1번문제 3-나) 풀이 코드
29 ENGLISH_FREQ = {
30     'A':8.17, 'B':1.50, 'C':2.78, 'D':4.25, 'E':12.70, 'F':2.23, 'G':2.02, 'H':6.09,
31     'I':6.97, 'J':0.15, 'K':0.77, 'L':4.03, 'M':2.41, 'N':6.75, 'O':7.51, 'P':1.93,
32     'Q':0.10, 'R':5.99, 'S':6.33, 'T':9.06, 'U':2.76, 'V':0.98, 'W':2.36, 'X':0.15,
33     'Y':1.97, 'Z':0.07
34 }
35
36 def calc_chi2(freq_pct):
37     return sum(
38         (freq_pct.get(c, 0) - ENGLISH_FREQ[c]) ** 2 / ENGLISH_FREQ[c]
39         for c in ENGLISH_FREQ
40     )
41
42 key = 0
43 temp_value = float('inf')
44 for k in range(26):
45     decrypt = caesar_dec(ct1, k)
46     freq = Counter(decrypt)
47     N = len(decrypt)
48     freq_pct = {c: round(cnt / N * 100, 2) for c, cnt in sorted(freq.items())}
49     chi2 = calc_chi2(freq_pct)
50     if chi2 < temp_value:
51         temp_value = chi2
52         key = k
53
54 print(f"\nkey : {key:02d} | 최소 chi2 : {temp_value:.4f}")

```

코드 4

key : 06 | 최소 chi2 : 4.6682

그림 4. 코드 4 실행 결과

3-다) 위에서 추측한 키 값으로 다음 암호문을 복호화하고 평문을 제시하시오.

3-나)를 통해 key를 6으로 추측하였다. 이를 이용하여 암호문을 복호화 하면 다음과 같은 의미를 파악할 수 있는 영문 평문을 얻을 수 있다.

```

56 # 1번문제 3-다) 풀이 코드
57 ciphertext = "ZNKYUBKXKOMTZEULZNXKXVAHROIULQJXKGYNGRRXKYVJOTZNVKVUVKGTJGRRYZGZKGZNUXOZEYNGRRKS
58 k = 6
59 plaintext = caesar_dec(ciphertext,k)
60 print(plaintext)

```

코드 5

THE SOVEREIGNTY OF THE REPUBLIC OF KOREA SHALL RESIDE IN THE PEOPLE AND ALL STATE AUTHORITY SHALL EMANATE FROM THE PEOPLE

그림 5. 코드 5 실행 결과

4. 비제네르 암호로 분류된 암호문의 키 길이를 추정하고 실제 키와 평문을 복원하시오.

4-가) 암호문에서 3글자 이상의 반복되는 패턴을 탐색하고, 반복 패턴 간의 위치 간격으로 키 길이를 추측하시오. 단, 키 길이는 5 이하이다.

비제네르 암호는 길이가 m 인 키를 반복해서 사용한다, 이러한 주기성으로 인해, 평문 내의 동일한 문자열 패턴이 키 길이 m 의 배수만큼 떨어진 위치에서 반복되면 암호문에서도 동일한 패턴으로 치환된다. 따라서 암호문에서 발견된 반복 패턴 사이의 거리 D 는 키 길이 m 의 배수일 가능성이 높다.

이러한 특성을 사용하여 비제네르 암호로 분류된 암호문 "ciphertexts2.txt"의 키 길이를 추정할 수 있다. 먼저 암호문 내에서 3글자 이상의 반복되는 패턴을 탐색하고 패턴 간의 거리 D 와 횟수를 구한다.

```

1 # 1번문제 4-가) 풀이 코드
2 from collections import defaultdict
3
4 def read_ciphertext(filepath):
5     with open(filepath, 'r', encoding='utf-8') as f:
6         content = f.read()
7         sentences = [s.strip() for s in content.split('\n') if s.strip()]
8         return [''.join(c for c in s.upper() if c.isalpha()) for s in sentences]
9
10 def kasiski(sentences):
11     distance_count = defaultdict(int)
12     for sentence in sentences:
13         for length in range(3, len(sentence)):
14             positions = defaultdict(list)
15             for start in range(len(sentence) - length + 1):
16                 positions[sentence[start:start+length]].append(start)
17             for pos_list in positions.values():
18                 if len(pos_list) >= 2:
19                     for i in range(1, len(pos_list)):
20                         distance_count[pos_list[i] - pos_list[i-1]] += 1
21     return dict(sorted(distance_count.items()))
22
23 ct2 = read_ciphertext(r'.\ciphertexts2.txt')
24 result = kasiski(ct2)
25 for dist, cnt in result.items():
26     print(f"거리 : {dist:3d} | 횟수 : {cnt}")

```

코드 6

거리 :	5		횟수 :	7
거리 :	8		횟수 :	1
거리 :	9		횟수 :	1
거리 :	10		횟수 :	22
거리 :	11		횟수 :	1
거리 :	13		횟수 :	1
거리 :	15		횟수 :	19

그림 6. 코드 6 실행 결과의 일부

문제 조건에 의해 추려진 키길이 후보인 2,3,4,5에 대해, 각 후보가 거리의 약수인 빈도를 조사하였다. 그 결과, 후보값중 5가 가장 높은 빈도를 기록하였고 이를 근거로 암호문의 키 길이를 5로 추정하였다.

```

28 for key_len in range(2, 6):
29     total = sum(cnt for dist, cnt in result.items() if dist % key_len == 0)
30     print(f"key_len : {key_len:3d} | 합계 : {total}")

```

코드 7

key_len :	2		합계 :	1845
key_len :	3		합계 :	462
key_len :	4		합계 :	454
key_len :	5		합계 :	2496

그림 7. 코드 7 실행 결과

4-나) 키 길이를 $|k|$ 라 할 때, 암호문을 $|k|$ 개의 부분열로 나눈다. 각 부분열은 동일한 단일 문자 치환으로 암호화되어 있다. 각 부분열에 대해 카이제곱 검정 또는 최빈 문자 추정법을 적용하여 각 위치의 키 k 를 구하시오.

4-가)에서 구한 바와 같이 $|k|=5$ 이다. 비제네르 암호는 키가 $|k|$ 자 주기로 반복 적용되는 특성을 가지므로, 전체 암호문을 5자 단위의 블록으로 분할하고 각 블록 내 동일한 위치에 대응하는 문자들을 추출하여 총 5개의 부분열을 구성한다.

이렇게 생성된 각 부분열은 동일한 단일 문자 치환으로 암호화된 상태이므로, 각 부분열은 시저 암호로 간주할

수 있다. 따라서 각 부분열에 대하여 영어 알파벳 빈도와의 카이제곱 독립성 검정을 수행하여 키를 확정할 수 있다.

```
ct2 = read_ciphertext(r'..\ciphertexts2.txt')
partition_sentences = partition(ct2)

keys = []
for sentence in partition_sentences:
    temp_value = float('inf')
    key = 0
    for k in range(26):
        decrypt = caesar_dec(sentence, k)
        freq = Counter(decrypt)
        N = len(decrypt)
        freq_pct = {c: round(cnt / N * 100, 2) for c, cnt in sorted(freq.items())}
        chi2 = calc_chi2(freq_pct)
        if chi2 < temp_value:
            temp_value = chi2
            key = k
    keys.append(key)
print(keys)
```

코드 8

[10, 11, 21, 14, 9]

그림 8. 코드 8 실행 과결

4-다) 나에서 추측한 키 값으로 다음 암호문을 복호화하고 평문을 제시하시오.

4-나)를 통해 구한 key로 주어진 암호문을 복호화 하였다. 복호화한 결과 다음과 같은 의미 있는 영어 평문을 얻을 수 있었다.

```
77 # 1번문제 4-다) 풀이 코드
78 def vigenere_dec(text, key):
79     key_len = len(key)
80     result = []
81     for i, c in enumerate(text):
82         result.append(chr((ord(c) - 65 - key[i % key_len]) % 26 + 65))
83     return ''.join(result)
84
85 ciphertext = "DSZGXFPMSRQYOMXPECSAOAPPUSNJTTCZOBRLGZAODDRNSYOVNZPJDUOLIRJVMNHJDPVICRZMWCIDCOUVP"
86 k = [10,11,21,14,9]
87 plaintext = vigenere_dec(ciphertext,k)
88 print(plaintext)
```

코드 9

THESOVEREIGNTYOFTHEREPUBLICOFKOREASHALLRESIDEINTHEPEOPLEANDALLSTATEAUTHORITYSHALLEMANATEFROMTHEPEOPLE

그림 9. 코드 9 실행 결과

5. 다음 4개의 문장은 시저암호와 비제네르 암호가 혼합되어 있다. 주어진 Ciphertext1.txt와 Ciphertext2.txt를 학습 데이터로하여 AI 분류기 모델을 생성하고, 아래의 암호문이 각각 어떤 유형으로 암호화 되었는지 구분하시오. 이때, 모델 및 데이터 학습 feature를 자세하게 기술하시오.

주어진 문제는 암호문을 시저암호와 비제네르암호로 분류하는 이진 분류 문제이다. 또한 주어진 학습 데이터가 모두 314문장으로 많지 않다. 위와 같은 점을 고려하여 적은 데이터로에서도 과적합 위험이 적고 안정적인 성능을 보이는 로지스틱회귀를 분류 모델로 선택하였다. 학습 feature로는 2-나)에서 살펴본 바와 같이 두 암호 체계 간의 뚜렷한 통계적 차이를 나타내는 일치지수를 선택하였다. 아래는 학습이 진행됨에 따라 Training, Validation의 변화 그래프이다.

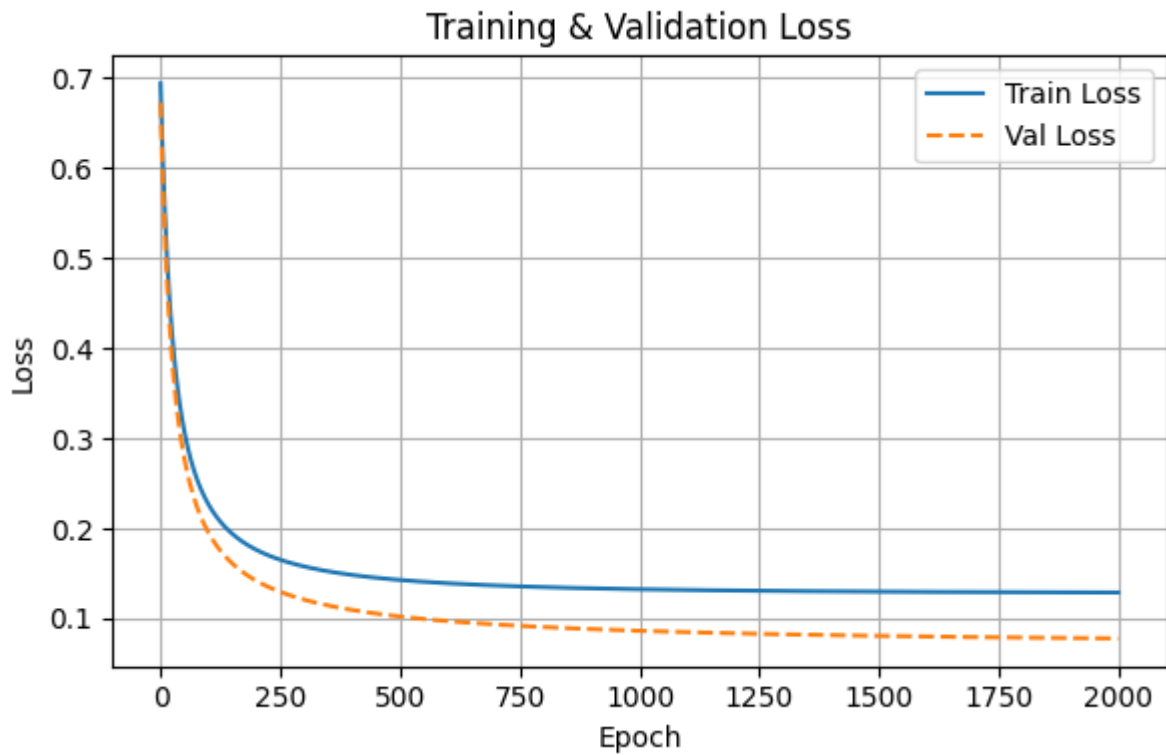


그림 10. 학습과정에서 Training, Validation loss의 변화

학습이 완료된 데이터를 사용하여 아래와 같은 결정 경계값을 얻을 수 있었다.

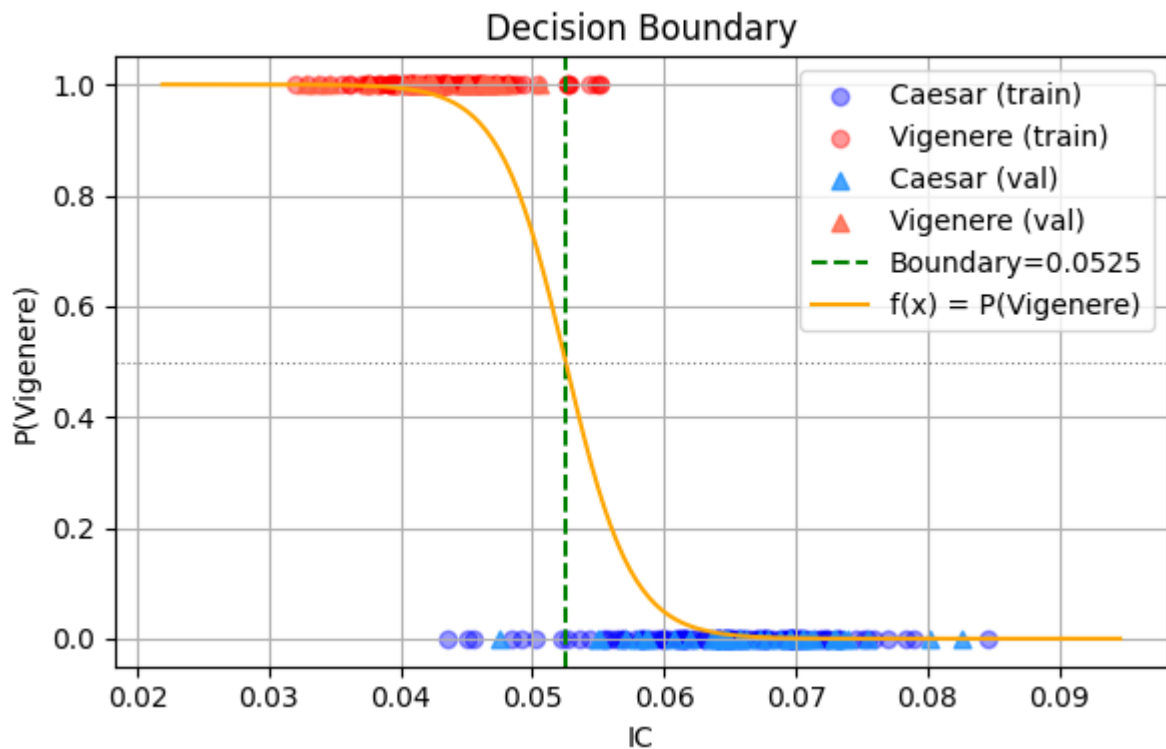


그림 11. Decision Boundary

해당 모델을 이용하여 주어진 4개의 암호문에 대해 어떤 유형으로 암호화 되었는지 분류하면 다음과 같이 모두 시저 암호화 되었음을 알 수 있다.

```
=== 분류 결과 ===  
암호문 1: IC=0.0626 P(Caesar)=0.9707 P(Vigenere)=0.0293 → Caesar  
암호문 2: IC=0.0639 P(Caesar)=0.9812 P(Vigenere)=0.0188 → Caesar  
암호문 3: IC=0.0540 P(Caesar)=0.6128 P(Vigenere)=0.3872 → Caesar  
암호문 4: IC=0.0540 P(Caesar)=0.6128 P(Vigenere)=0.3872 → Caesar
```

그림 12. 분류 결과

본 문제의 분류 모델 코드 및 실행문은 1번_5.py에서 확인할 수 있다.